Explain, Then Forget: Recommendation Unlearning under Asymmetric Access Constraint

Haichao Zhang[1,2], Chong Zhang[1,2], Wen Wang[1], Shi Qiu[1], Peiyu Hu[1,2], Jia Wang[1,2†]
[1]Xi'an Jiaotong-Liverpool University, China
[2]University of Liverpool, United Kingdom
Email: haichao.zhang22@student.xjtlu.edu.cn, jia.wang02@xjtlu.edu.cn

***Abstract*—Recommendation unlearning aims to remove the influence of specific historical interactions that should no longer affect future recommendations, such as noisy clicks or outdated user preferences. However, performing recommendation unlearning becomes much harder in modern Machine-Learning-as-a-Service (MLaaS) architectures, where the recommendation model is remotely hosted by a model provider (MP) and accessed by service operators (SO) only through APIs. In this setting, a service operator serves end users but cannot access or modify internal model parameters. As a result, existing unlearning methods are difficult to apply because they require access to model internals for retraining, fine-tuning, or other parameter updates. To address these challenges, we propose Causal Explanation-based Unlearning (CEU), a novel unlearning framework tailored for MLaaS environments. Instead of modifying internal model weights, CEU operates through a two-step output-level score intervention. First, it estimates how the forgotten interaction causally contributes to the current recommendation. Second, guided by this causal signal, the system removes the influence of the forgotten interaction by adjusting the final ranking scores of affected items. Because CEU leaves model parameters untouched, it naturally supports continuous unlearning requests while avoiding the error accumulation that can result from repeated parameter updates. Extensive experiments on three public datasets show that CEU achieves unlearning effectiveness comparable to full retraining while preserving recommendation utility and significantly reducing computational cost. The source code is available at: https://github.com/zhang-haichao/Explain-and-Forget.**

***Index Terms*—Recommendation Unlearning, Causal Inference, Counterfactual Explanation, Graph Neural Networks.**

I. Introduction

Driven by growing model complexity and the high cost of training and maintenance, modern recommender systems increasingly adopt the Machine Learning as a Service (**MLaaS**) architecture [1]–[3]. In this setting, the model provider (**MP**) trains and hosts the recommendation model, while the service operator (**SO**) serves end users by querying the model through APIs [4]. This separation means that the SO can use the model but cannot access or modify its internal parameters. As the direct interface with users, the SO is responsible for maintaining the quality of recommendation results. However, user preferences change over time. Users may lose interest in previously preferred items or accidentally click irrelevant ones. Such noisy or outdated interactions can cause the system to repeatedly recommend unwanted content [5]–[7]. To maintain user engagement, the SO should allow users to revoke specific historical interactions and remove their influence on future recommendations. This practical need motivates the task of recommendation unlearning [8], which aims to remove the influence of selected historical interactions from a trained recommendation model.

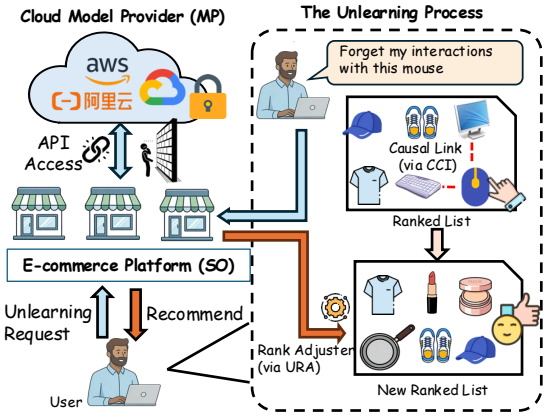


Fig. 1. Illustration of recommendation unlearning in MLaaS. Since small E-commerce Platforms (SO) often lack the resources to train complex models, they rely on Cloud Model Providers (MP) to train models and serve recommendation results via APIs. To remove the influence of specific interactions, CEU uses CCI to identify causal contributions and uses URA to adjust rankings without modifying MP parameters.

Achieving recommendation unlearning in the MLaaS setting introduces two critical. First, a strict asymmetric access constraint: the SO has no access to the recommendation model's internal parameters or gradients. Since the model is controlled by the MP and exposed only through APIs, the SO cannot directly modify model weights to remove the effect of specific training data. Second, unlearning requests may arrive continuously as users interact with the system [9], [10]. Repeated approximate updates across sequential requests may accumulate errors, gradually degrading overall recommendation quality [11]. These two constraints make existing unlearning methods difficult to apply, because most of them require access to model internals or control over the training process. Existing methods can

† Corresponding author.

This study was funded by the National Natural Science Foundation of China (Grant No. 72401232), the Natural Science Foundation of the Jiangsu Higher Educational Institution of China (Grant No. 23KJB520037), the XJTLU Research Development Fund (RDF-21-01-053), and Liaoning Liaohe Lab (Grant Nos. LLL24ZZ-02-01 and LLL24ZZ-02-02).

be broadly grouped into three categories: **retraining-based methods** (e.g., GraphEraser [12], RecEraser [13]) that retrain affected model partitions; **fine-tuning-based methods** (e.g., GNNDelete [14], UnlearnRec [15]) that apply lightweight representation updates; and **influence-based methods** (e.g., SCIF [16], IFRU [17]) that adjust weights based on estimated data influence. Despite their different techniques, these approaches share a common limitation: they require access to model internals, training procedures, or updatable model components. As a result, they are difficult to apply in MLaaS systems where the model can only be accessed through APIs.

Given the strict constraints of the MLaaS paradigm, recommendation unlearning should shift away from parameter updates, as the Service Operator (SO) can only observe outputs via APIs. This suggests a different unlearning strategy: instead of updating model parameters, the system should revise recommendation outputs to remove the effect of forgotten interactions. However, executing output-level unlearning is non-trivial and presents two major challenges: (1)**Which items to adjust?** To revise outputs correctly, the system must first identify which candidate items in the recommendation list were actually influenced by the forgotten interaction. (2)**How much to adjust?** Once the affected items are identified, the system must carefully penalize their scores to erase the forgotten data's influence. This adjustment must be highly precise. If the score reduction is too aggressive, it will distort the user's normal preferences; if it is too weak, the model will fail to truly *forget* the revoked interaction, leaving unwanted data traces behind.

To address these challenges, we propose Causal Explanation-based Unlearning (**CEU**), a novel framework (illustrated in Fig. 1) for recommendation unlearning through output-level score intervention in MLaaS environments, without modifying internal model parameters. To address the first challenge, our framework employs a Causal Contribution Identifier (**CCI**). This module identifies the historical interactions that causally support a recommendation and estimates their contribution strengths. Instead of exposing internal model parameters, the MP only needs to provide these attribution scores to the SO through the API interface. This design remains compatible with the MLaaS access setting without exposing model internals. To address the second challenge, we introduce the Unlearning-driven Rank Adjuster (**URA**). Guided by the causal contributions provided by the CCI, the SO uses this module to adjust the scores of affected items and remove the influence of the forgotten interaction. Because CEU does not modify model parameters, it is naturally suited to continuous unlearning requests and avoids the error accumulation caused by repeated parameter updates. Appendix results under increasing forgetting ratios further support this property. Our main contributions are summarized as follows:

- We propose CEU, a novel recommendation unlearning framework designed for MLaaS environments. CEU removes the influence of forgotten interactions through output-level score intervention without modifying internal model parameters.
- We design the **Causal Contribution Identifier (CCI)** to identify the causal drivers of recommendations, thereby solving the attribution challenge in complex models.
- We introduce the **Unlearning-driven Rank Adjuster (URA)**, which removes the influence of forgotten interactions by applying localized score adjustments guided by the causal contributions identified by CCI.
- Extensive experiments on three public datasets show that CEU achieves unlearning effectiveness comparable to full retraining while preserving recommendation utility and significantly reducing computational cost.

II. Related Work

A. Recommendation Unlearning

Recommendation unlearning seeks to eliminate the influence of selected interactions from trained recommender models, e.g., removing outdated or noisy behaviors. Existing approaches can be broadly grouped into three categories. First, retraining-based methods (e.g., GraphEraser [12], RecEraser [13]) aim for complete removal by partitioning data and retraining affected sub-models (SISA [18]). While exact, this paradigm severely limits efficiency and scalability. Second, fine-tuning-based methods (e.g., GNNDelete [14], UnlearnRec [15]) avoid full retraining by applying lightweight representation updates to correct the influence of the forgotten data. Third, influence-based methods (e.g., SCIF [16], IFRU [17]) efficiently approximate the gradient or loss impact of deleted data to adjust model weights. Furthermore, prompt-tuning-based recommendation unlearning has also been explored, where lightweight prompts and a teacher–student objective are used to forget target interactions while preserving useful retained knowledge (e.g., Prompt-Tuning for Recommendation Unlearning [19]). Despite their efficiency, both fine-tuning and influence-based approximate methods often suffer from incomplete forgetting, unintended performance drift, and accumulated errors during continuous sequential requests. Crucially, because all three paradigms require accessing model internals to update parameters, they are incompatible with the strict asymmetric access constraints of MLaaS. This motivates our parameter-free framework.

B. Causal Explainable Recommendation

Recent studies use causal and counterfactual perspectives to identify what truly drives a recommendation, generally falling into two directions. The first builds causality-aware models incorporating explicit reasoning structures. For example, using reinforcement learning on knowledge graphs [20] or neural symbolic reasoning (CAFE [21]) to trace underlying user intents. The second focuses on post-hoc counterfactual explanations, analyzing trained black-box recommenders by asking "what-if" questions. CountER [22] searches for minimal input changes reversing a decision, while CF^2 [23] combines factual and counterfactual reasoning to capture necessity and sufficiency. While these methods quantify causal influence for interpretation and transparency, they do not leverage these

attributions to execute recommendation unlearning, especially under the strict constraints of MLaaS architectures.

III. Preliminaries

A. Recommendation Task

Let $\mathcal{U} = \{u_1, u_2, ...\}$ denote the set of users and $\mathcal{I} = \{i_1, i_2, ...\}$ denote the set of items. The historical user-item interactions are represented by the dataset $\mathcal{D} = \{(u, i) \mid u \in \mathcal{U}, i \in \mathcal{I}, R_{ui} = 1\}$, where $R_{ui} = 1$ indicates an observed interaction. The training set is denoted as $\mathcal{D}_{\text{tr}} \subseteq \mathcal{D}$.

The primary goal in recommendation is to train a model f with parameters θ using a learning algorithm A, such that $\theta = A(\mathcal{D}_{\text{tr}})$. This model f is used to predict preference scores $\hat{y}_{ui} = f(u, i \mid \theta)$ for unobserved user-item pairs, from which a personalized top-K ranking list is generated for each user.

B. Recommendation Unlearning

Recommendation unlearning aims to efficiently eliminate the influence of specific data from a pre-trained model f. Formally, let $\mathcal{D}_{\text{tr}}$ be the original training set. We define a "forgotten set" $\mathcal{D}_f \subseteq \mathcal{D}_{\text{tr}}$ as the collection of user-item interactions that must be removed. Correspondingly, the "retained set" is defined as $\mathcal{D}_r = \mathcal{D}_{\text{tr}} \setminus \mathcal{D}_f$.

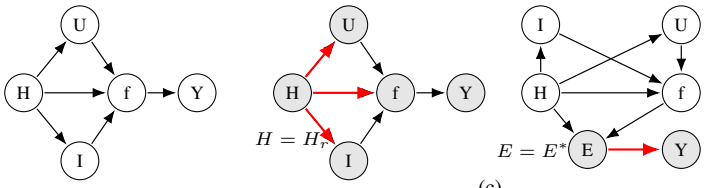
The ideal but computationally prohibitive solution is to obtain a "gold model," f_r, by retraining from scratch using only the retained data. The parameters of this model are denoted as $\theta_r = A(\mathcal{D}_r)$. Given the infeasibility of retraining, the practical task is to design an efficient unlearning method that approximates the recommendation behavior of this gold model. Parameter-based approaches achieve this by updating model parameters to obtain an unlearned model f_u with weights θ_u. In contrast, parameter-free methods like CEU perform unlearning through output-level adjustment without modifying the original θ.

The objective is to design U such that f_u closely approximates the gold model f_r, while balancing three primary goals:

- **G1: Unlearning Completeness.** The unlearned model f_u must completely eliminate the influence of the forgotten data $\mathcal{D}_f$.
- **G2: Unlearning Efficiency.** The unlearning algorithm U must be time-efficient, avoiding the high computational cost.
- **G3: Model Utility.** The unlearned model f_u must maintain recommendation performance comparable to the gold model f_r.

C. Unlearning as Causal Intervention

We use a Structural Causal Model (SCM) framework [24] as a conceptual view of recommendation unlearning and to motivate our intervention design. Fig. 2 (a) illustrates the standard recommendation SCM, where historical interactions (H) causally influence the recommendation output (Y) through multiple paths. These include a "front-door" path $H \to f \to Y$, as well as "back-door" paths mediated by user and item representations, $H \to U \to f \to Y$ and $H \to I \to f \to Y$.



(a) Standard recommendation. (b) Retraining-based unlearning. (c) Causal-intervention unlearning.

Fig. 2. Causal graphs of (a) standard recommendation, (b) retraining-based unlearning via $do(H := H_r)$, and (c) CEU, which intervenes on $E \to Y$ to suppress forgotten-data effects without updating f. H, U, I, f, E, and Y denote history, users, items, model, explanation, and output, respectively.

Fig. 2 (b) illustrates retraining-based unlearning as a global intervention on the node H, which can be described by the do-operator as:

$$Y_r = P(Y \mid do(H := H_r)),$$

where H_r denotes the remaining historical interactions after removing the forgotten subset H_f from H. This intervention (red arrows) invalidates all downstream descendants (gray nodes U, I, f, Y), forcing their complete recomputation. While this "full-graph" intervention is causally complete (satisfying G1 and G3), it requires high-cost recomputation, thus violating G2.

As shown in Fig. 2 (c), our approach approximates unlearning through a localized intervention guided by causal explanation. We augment the SCM by introducing an explanation node:

$$E = \text{explain}(H, f),$$

which represents the counterfactual contribution of H to Y. This redirects the causal source of Y: we *prune* the original causal edge $f \to Y$ and *graft* a new path $E \to Y$ (red arrow). Without modifying any model parameters, we perform a localized intervention along this path:

$$Y_r = P(Y \mid \text{do}(E := E^*)),$$

where E^* denotes the updated explanatory representation after removing the forgotten interaction. This design is causally motivated rather than fully identified in the formal sense. It treats the explanation path as a practical approximation of how forgotten interactions affect the output and intervenes only in the part of the ranking process most directly linked to the forgotten effect, without changing global model parameters. In this way, it provides an efficient and interpretable approximation to recommendation unlearning, with the goal of balancing G1, G2, and G3 in practice.

IV. Proposed Methods

A. Overview

We propose Causal Explanation-based Unlearning (CEU), a unified framework that performs recommendation unlearning through a causal "explain-then-forget" paradigm. The central idea is that unlearning should not rely on direct parameter

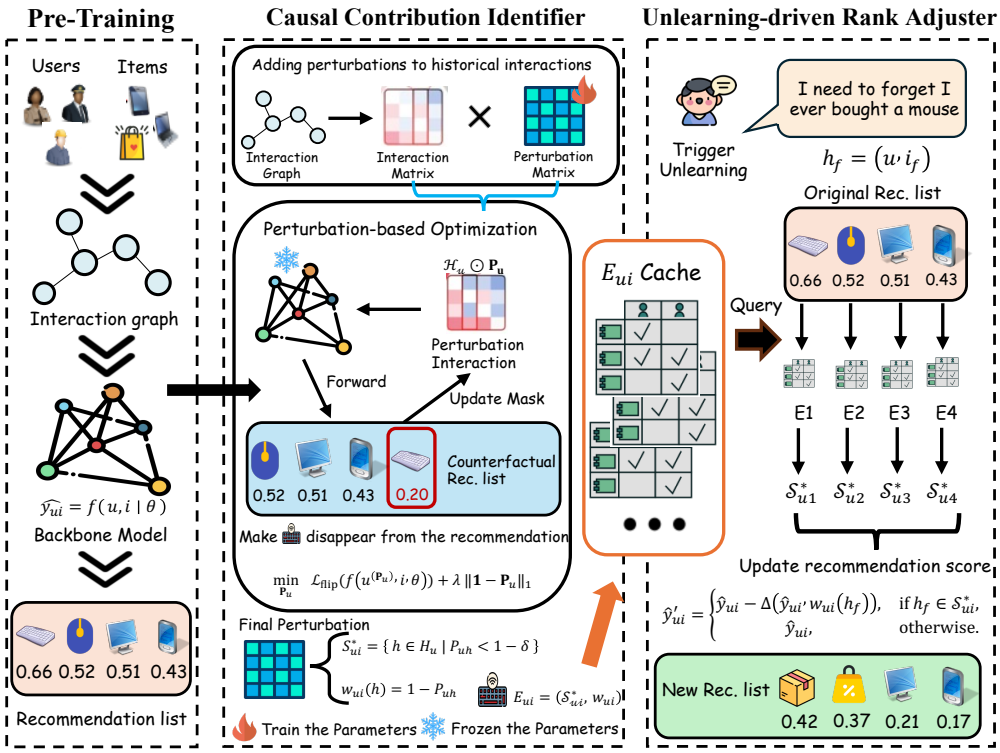


Fig. 3. The framework of CEU. The **Pre-Training** pipeline generates the initial ranking results. The **Causal Contribution Identifier** identifies the minimal causal support set and quantifies their contribution to the recommendation. The **Unlearning-driven Rank Adjuster** performs localized score adjustment to remove the influence of the forgotten interaction.

modification but instead be guided by causal reasoning: before removing the influence of specific data, the system must understand why that data affects the recommendation outcome. As illustrated in Fig. 3, CEU comprises two sequential modules:

- **Causal Contribution Identifier (CCI) — The Explain Stage.** In our MLaaS setting, the Cloud Model Provider (MP) hosts the CCI module. Since the MP has full access to the model, it directly computes the causal influence of past interactions and shares these attributions with the SO via a dedicated API. Specifically, for a user u and a recommended item i, the CCI analyzes the user's interaction history H_u. It first identifies the Minimal Causal Support Set ($S_{ui} \subseteq H_u$), which contains the key interactions that primarily drive the recommendation. Then, it calculates the causal contribution strength, $w_{ui}(h)$, for each historical item $h \in H_u$.
- **Unlearning-driven Rank Adjuster (URA) — The Forget Stage.** When requested to unlearn a specific interaction $h_f = (u, i_f)$, URA uses the causal contribution weights provided by CCI. It performs a targeted intervention at the post-hoc ranking stage by adjusting the scores of candidate items. This precisely removes the influence attributed to i_f along the identified causal path ($E \to Y$ in Fig. 2(c)), thereby mitigating side effects and achieving high efficiency and locality.

B. Causal Contribution Identifier

The Causal Contribution Identifier (CCI) module serves as the *Explain* component within the CEU framework. It aims to uncover why a specific item i is recommended to user u by identifying the minimal set of historical interactions whose removal would reverse the recommendation decision.

1) Explanation of recommendation: Inspired by counterfactual reasoning [22], CCI formulates the explanation generation process as searching for the smallest perturbation to the user's history that flips the model's output. Let the historical interactions for user u be denoted as:

$$\mathcal{H}_u = \{(u, i_j) \mid (u, i_j) \in \mathcal{D}_{\text{tr}}\}.$$

We define $\mathcal{S}^*_{ui}$ as the Causal Support Set, representing the minimal subset of the user's historical interactions that are necessary and sufficient to cause the recommendation of item i.

$$\mathcal{S}^*_{ui} = \arg\min_{\mathcal{S} \subseteq \mathcal{H}_u} |\mathcal{S}| \quad \text{s.t.} \quad \|f(u, i|\mathcal{H}_u, \theta) - f(u, i|\mathcal{H}_u \backslash \mathcal{S}, \theta)\| > \epsilon$$

where ϵ is a threshold defining causal sensitivity. We define the explanation for recommendation (u, i) as a tuple:

$$E_{ui} = (\mathcal{S}^*_{ui}, w_{ui}),$$

where $\mathcal{S}^*_{ui}$ is the Causal Support Set, and $w_{ui}(h)$ is the *Contribution Function* measuring the causal strength of each historical interaction $h \in \mathcal{S}^*_{ui}$. This formulation captures the counterfactual logic: *"If the interactions in $\mathcal{S}^*_{ui}$ had not occurred, item i would not have been recommended to user u."*

2) Quantifying Causal Contributions: We obtain the explanation E_{ui} by learning a minimal perturbation on user u's history that reverses the recommendation of item i. We introduce a continuous mask vector $\mathbf{P}_u \in [0,1]^{|\mathcal{H}_u|}$ applied to the user's interaction history $\mathcal{H}_u$, where each element P_{uh} corresponds to one specific interaction $h \in \mathcal{H}_u$. The value P_{uh} controls the retention degree of interaction h: a value of 1 means the interaction is fully preserved, while smaller values indicate the interaction is suppressed or removed. Thus, $\mathbf{P}_u$ defines a weighted version of the user history, the masked history is $\mathcal{H}_u \odot \mathbf{P}_u$.

Following the Occam's Razor principle [25], we aim for explanations that are both simple and effective. To obtain such an explanation, we solve the following optimization problem:

$$\min_{\mathbf{P}_u} \ \mathcal{L}_{\text{flip}}\big(f(u^{(\mathbf{P}_u)}, i \mid \theta)\big) + \lambda \, \|\mathbf{1} - \mathbf{P}_u\|_1,$$

where $u^{(\mathbf{P}_u)}$ denotes the counterfactual user obtained by weighting the historical interactions of user u with the mask vector $\mathbf{P}_u$, and $\mathcal{L}_{\text{flip}}$ enforces a significant decrease in the predicted score for item i, and the ℓ_1 term encourages a sparse perturbation (few interactions altered). A practical choice for the flip loss uses a hinge-margin against the $(K+1)$-th item i_{K+1} in u's original list, where K is the cutoff of the top-K recommendation list for user u, and i_{K+1} denotes the item ranked $(K+1)$-th in the original ranking.

$$\mathcal{L}_{\text{flip}} = \max\big(0,\, m + f(u^{(\mathbf{P}_u)}, i \mid \theta) - f(u, i_{K+1} \mid \theta)\big),$$

with margin $m > 0$. Since the MP executes the CCI module, it has full access to the internal parameters θ. Therefore, the MP can directly optimize the mask P_u using standard gradient-based updates, while keeping the model weights strictly frozen. After optimization, interactions with $P_{uh} < 1 - \delta$ (a small threshold δ) are deemed causal. The resulting *Causal Support Set* is

$$\mathcal{S}^*_{ui} = \{\, h \in \mathcal{H}_u \mid P_{uh} < 1 - \delta \,\},$$

which represents the minimal subset of the user's historical interactions that are necessary and sufficient to cause the recommendation of item i. We define the per-interaction *causal contribution weight* as

$$w_{ui}(h) = 1 - P_{uh},$$

which quantifies the strength of h in sustaining $\hat{y}_{ui} = f(u, i \mid \theta)$. The explanation for (u, i) is $E_{ui} = (\mathcal{S}^*_{ui},\, w_{ui})$, which will be used by the unlearning module.

C. Unlearning-driven Rank Adjuster

The Unlearning-driven Rank Adjuster (URA) module executes the *Forget* phase of the CEU framework. It performs efficient and precise recommendation unlearning based on the causal attributions generated by the CCI module (Section IV-B). Its objective is to remove the influence of a forgotten interaction (u, h_f) without retraining or altering model parameters. URA achieves this by locally adjusting the ranking scores of items affected by the forgotten cause.

1) Causal-Guided Unlearning: When a forgetting request is received for a specific user–interaction pair $h_f = (u, i_f)$, URA first identifies the potential propagation of its causal effect. For each candidate item i in the user's current recommendation list $L_u = \{i_1, i_2, \ldots, i_K\}$, the SO queries the MP's attribution API to retrieve the causal explanations $E_{ui} = (\mathcal{S}^*_{ui}, w_{ui})$. This lightweight interaction allows the SO to obtain the necessary causal guidance without accessing any internal model parameters. Here, $\mathcal{S}^*_{ui}$ is the Causal Support Set describing which historical interactions of user u are causally responsible for recommending i, and $w_{ui}(h)$ quantifies the causal contribution of each interaction $h \in \mathcal{S}^*_{ui}$. These explanations can be precomputed and cached for real-time unlearning.

2) Localized Score Adjustment: URA adjusts the predicted preference scores based on the causal relevance of the forgotten interaction. Let $\hat{y}_{ui} = f(u, i \mid \theta)$ denote the original prediction for user u and item i. The adjusted score $\hat{y}'_{ui}$ is computed as:

$$\hat{y}'_{ui} = \begin{cases} \hat{y}_{ui} - \Delta(\hat{y}_{ui}, w_{ui}(h_f)), & \text{if } h_f \in \mathcal{S}^*_{ui}, \\ \hat{y}_{ui}, & \text{otherwise,} \end{cases}$$

where only items whose recommendations are causally dependent on h_f are modified, ensuring that interventions remain local and precise.

The penalty function $\Delta(\cdot, \cdot)$ determines the degree of attenuation according to the causal contribution $w_{ui}(h_f)$. A simple yet effective form is the multiplicative penalty:

$$\Delta(\hat{y}_{ui}, w_{ui}(h_f)) = \alpha \cdot \hat{y}_{ui} \cdot w_{ui}(h_f),$$

which proportionally reduces $\hat{y}_{ui}$ based on the strength of causal dependence. A higher $w_{ui}(h_f)$ leads to a larger decrease in $\hat{y}_{ui}$, reflecting stronger causal influence.

3) Reconstruction of the Recommendation Ranking: After computing the adjusted scores $\{\hat{y}'_{ui} \mid i \in L_u\}$ for all candidate items, URA reconstructs the user's recommendation list by re-ranking items according to $\{\hat{y}'_{ui}\}_{i \in L_u}$. The final unlearned ranking is denoted as:

$$L'_u = \text{Rank}\big(\{\hat{y}'_{ui}\}_{i \in L_u}\big).$$

This operation effectively removes the causal effect of h_f from user u's recommendations while preserving unaffected preferences. Because CEU does not update model parameters, it naturally avoids the error accumulation that may arise in sequential unlearning requests. Additional results under increasing unlearning ratios are reported in the Appendix.

V. Experiments

We conduct extensive experiments on three public datasets to evaluate the proposed CEU framework, aiming to answer the following four research questions:

- **RQ1: Utility Preservation (G3).** Can CEU preserve recommendation utility on the retained data while enforcing unlearning, while remaining close to the original model and the retrained reference?

TABLE I

RECOMMENDATION UTILITY AFTER UNLEARNING UNDER A 1% FORGETTING RATIO. ORIGIN AND RETRAIN ARE REFERENCE RESULTS ONLY AND ARE EXCLUDED FROM METHOD COMPARISON; BOLD DENOTES THE BEST PRACTICAL UNLEARNING METHOD. RESULTS ARE AVERAGED OVER 3 RUNS ($p < 0.05$, PAIRED T-TEST).

ML-1M	**BPR**				**LightGCN**			
	HR@10	**NDCG@10**	**HR@20**	**NDCG@20**	**HR@10**	**NDCG@10**	**HR@20**	**NDCG@20**
Origin	0.7520	0.2526	0.8444	0.2597	0.7680	0.2715	0.8637	0.2787
Retrain	0.7419	0.2417	0.8488	0.2525	0.7649	0.2672	0.8613	0.2696
GraphEraser	0.4540	0.0902	0.6126	0.0995	0.5635	0.1295	0.6968	0.1388
RecEraser	0.4833	0.1055	0.6260	0.1123	0.6003	0.1467	0.7379	0.1574
SCIF	0.5341	0.1332	0.6833	0.1436	0.6060	0.1723	0.7230	0.1764
IFRU	0.4227	0.2248	0.5749	**0.2634**	0.4350	0.2205	0.6502	**0.2746**
GNNDelete	0.6088	0.1212	0.7897	0.1511	0.5139	0.1305	0.6375	0.1390
UnlearnRec	0.5784	0.2206	0.7396	0.2318	0.4937	0.1320	0.6219	0.1415
CEU	**0.7373**	**0.2533**	**0.8364**	0.2602	**0.7584**	**0.2671**	**0.8535**	0.2736
Netflix	**BPR**				**LightGCN**			
	HR@10	**NDCG@10**	**HR@20**	**NDCG@20**	**HR@10**	**NDCG@10**	**HR@20**	**NDCG@20**
Origin	0.8550	0.2365	0.9453	0.2024	0.8647	0.2418	0.9497	0.2043
Retrain	0.7805	0.1631	0.9051	0.1750	0.8122	0.2020	0.9226	0.1930
GraphEraser	0.6272	0.1005	0.8199	0.0938	0.7118	0.1225	0.8563	0.1165
RecEraser	0.6795	0.1079	0.8225	0.1021	0.7228	0.1319	0.8880	0.1266
SCIF	0.4020	0.0748	0.5239	0.0829	0.4825	0.0935	0.6377	0.1045
IFRU	0.2037	0.0938	0.2997	0.1192	0.2556	0.1373	0.3452	0.1744
GNNDelete	0.7515	0.1622	0.8879	0.1703	0.5473	0.1690	0.6932	0.1785
UnlearnRec	0.7267	0.1641	0.8667	0.1751	0.6318	0.1670	0.7192	0.1715
CEU	**0.8527**	**0.2343**	**0.9437**	**0.2006**	**0.8627**	**0.2402**	**0.9480**	**0.2031**

- **RQ2: Unlearning Effectiveness (G1).** Does CEU successfully eliminate the influence of forgotten interactions from future recommendations?
- **RQ3: Time Efficiency (G2).** What is the end-to-end latency and computational cost of CEU under both single and batched unlearning requests?
- **RQ4: Ablation on CCI.** How crucial is the causal attributions step (CCI) to the overall effectiveness of CEU?

A. Experiment Setup

1) Datasets: We evaluate CEU on three public benchmarks (Table II): ML-100K[1], ML-1M, and Netflix[2]. After binarizing the feedback, we first split the dataset into training, validation, and test sets with a ratio of $70/10/20$. Then, we randomly sample 10% of the interactions from the training set to form the forgetting set $\mathcal{D}_f$, leaving the rest of the training data as the retained set $\mathcal{D}_r$.

TABLE II

STATISTICS OF THREE PUBLIC DATASETS.

Dataset	Users	Items	Interactions	Avg. Inter.	Sparsity
ML-100K	944	1,683	100,000	106	93.71%
ML-1M	6,041	3,707	1,000,209	165	95.53%
Netflix	15,060	13,352	2,545,835	245	91.83%

2) Baselines: We evaluate CEU on two backbone recommenders, BPR [26] and LightGCN [27], to cover both matrix-factorization-based and graph-based settings. We compare CEU with representative unlearning methods from three categories.

- **Retraining-based methods.** Retrain removes the forgotten data and retrains the model from scratch on the retained set, serving as the gold standard. GraphEraser [12] and RecEraser [13] reduce retraining cost by restricting updates to affected graph partitions or data shards.
- **Fine-tuning-based methods.** GNNDelete [14] and UnlearnRec [15] perform unlearning by updating lightweight components during the unlearning stage, so that the representations influenced by removed interactions can be revised.
- **Influence-based methods.** SCIF [16] and IFRU [17] estimate the influence of removed data and apply corrective parameter updates without full retraining.

All methods are evaluated under the same data split and forgetting setting for fair comparison.

3) Evaluation Metrics: We evaluate recommendation utility with HR@K and NDCG@K for $k \in \{5, 10\}$ [28], [29], where higher values indicate better retained-data performance. We evaluate unlearning completeness with Membership Inference Attack (MIA) accuracy [30]; values closer to 0.5 indicate stronger forgetting. We evaluate efficiency by measuring the end-to-end latency of unlearning requests. More metric details are provided in Appendix.

B. Performance Comparison (RQ1)

We evaluate utility preservation under a 1% forgetting ratio on two datasets with BPR and LightGCN, reporting HR@K and NDCG@K ($K \in \{10, 20\}$) as in Table I. Here, Origin denotes the original model before unlearning, while Retrain denotes the gold-standard model retrained on the retained data; both are reported only as reference results and are not included in the comparison among practical unlearning methods. CEU consistently surpasses approximate baselines,

[1]https://grouplens.org/datasets/movielens/

[2]http://netflixprize.com

TABLE III

RESULTS OF UNLEARNING COMPLETENESS EVALUATED BY MEMBERSHIP INFERENCE ATTACK (MIA). ACC@1 REPRESENTS ACCURACY WITH 1% FORGETTING, WHILE ACC@10 REPRESENTS ACCURACY WITH 10% FORGETTING. ATTACKER ACCESS: BOTH THE ORIGINAL SCORE $\hat{y}_{ui}$ AND THE ADJUSTED SCORE $\hat{y}'_{ui}$ ARE AVAILABLE TO THE ATTACKER.

Dataset	Metric	LightGCN			BPR		
		Origin	Retrain	CEU	Origin	Retrain	CEU
ML-100K	ACC@1	0.9560	0.5342	0.7370	0.9797	0.5380	0.8476
	ACC@10	0.9581	0.6567	0.5155	0.9801	0.6556	0.6429
ML-1M	ACC@1	0.9678	0.5126	0.4789	0.9619	0.5124	0.6847
	ACC@10	0.9801	0.5015	0.6429	0.9626	0.6443	0.5083
Netflix	ACC@1	0.9885	0.6538	0.5581	0.9819	0.6538	0.5581
	ACC@10	0.9955	0.7059	0.5000	0.9920	0.6029	0.5089

TABLE IV

PRECOMPUTATION COST AND CACHE FOOTPRINT OF CCI (FP32). WE REPORT DATASET-LEVEL TOTALS AND PER-USER AVERAGES IN PARENTHESES.

Dataset	Forgotten Users	Time (total)	Cache per-User	Cache Size (Total)
ML-100K	9	3.10 s/user (28.30s)	22.45 KiB	202.05 KiB
ML-1M	60	10.10 s/user (600.64s)	22.75 KiB	1365.00 KiB
Netflix	30	8.40 s/user (240.12s)	21.88 KiB	656.40 KiB

nearing original performance across all settings. On ML-1M with LightGCN, CEU achieves HR@10 = 0.7584 and NDCG@10 = 0.2671, with minimal drops from origin's 0.7603 and 0.2682 (0.25% and 0.41% reductions), outperforming SCIF (HR@10 = 0.6060) by 20.29% and NDCG@10 by 21.14%. Retraining-based methods like RecEraser (HR@10 = 0.6003) and GraphEraser (HR@10 = 0.5336) lag significantly (21.00% and 29.64% below CEU on ML-1M LightGCN) due to data fragmentation. Similarly, fine-tuning-based methods (e.g., GNNDelete) also suffer from utility degradation, while CEU's targeted intervention preserves utility effectively. This trend holds across datasets. These results confirm that CEU successfully preserves model utility (G3), avoiding the significant collateral damage to non-target data that plagues other approximate and SISA-based methods. Additional results in the Appendix further show that, as the forgetting ratio increases, baseline methods exhibit clear degradation on the retained set, while CEU remains much more stable.

C. Unlearning Completeness (RQ2)

Following prior work [16], [31], we employ a membership inference attack to assess the extent to which the forgotten data has been removed. For this attack, the adversary is assumed to have access to both the model's raw prediction logits ($\hat{y}_{ui}$) and the final post-processed recommendation scores ($\hat{y}'_{ui}$). All reported attack results are based on this full-knowledge white-box threat model. We report the attacker's accuracy (ACC) at 1% and 10% forgetting ratios in Table III. To rigorously evaluate absolute completeness, we directly benchmark CEU against the unmitigated Origin model and the ideal Retrain standard. CEU significantly reduces MIA accuracy, approaching Retrain's ideal forgetting performance while outperforming the unmitigated Origin model. For instance, on ML-1M with LightGCN, CEU achieves ACC@1 = 0.4789 and ACC@10 = 0.6429, a substantial drop from Origin's 0.9678 and 0.9801 (50.53% and 34.42% reductions), and closely aligns with Retrain's 0.5126 and 0.5015. On Netflix with LightGCN, CEU's ACC@10 = 0.5000 nearly matches random guessing, compared to Origin's 0.9955, highlighting its superior forgetting efficacy. These results confirm that CEU effectively erases forgotten data, fulfilling the Unlearning Completeness (G1) objective.

TABLE V

UNLEARNING LATENCY (S). BOLD AND UNDERLINED VALUES INDICATE THE BEST AND SECOND-BEST RESULTS, RESPECTIVELY; SPEEDUP IS MEASURED AGAINST THE SECOND-BEST METHOD.

Method	ML-100K		ML-1M		Netflix	
	BPR	LightGCN	BPR	LightGCN	BPR	LightGCN
Retrain	248.37	141.92	1935.41	4209.05	527.66	1019.13
GraphEraser	26.08	<u>17.93</u>	302.11	269.74	310.06	640.82
RecEraser	29.31	35.19	386.57	275.02	404.68	880.21
SCIF	<u>18.36</u>	18.50	66.72	64.09	299.88	177.14
IFRU	55.07	57.63	78.40	90.55	<u>104.12</u>	<u>117.95</u>
GNNDelete	32.94	35.39	68.27	67.79	152.38	177.17
UnlearnRec	19.21	20.03	<u>51.68</u>	<u>59.32</u>	113.58	123.16
CEU	**0.36**	**0.34**	**4.50**	**6.37**	**2.81**	**2.89**
Speedup	51.00×	52.74×	11.48×	9.31×	37.05×	40.81×

D. Unlearning Efficiency (RQ3)

We evaluate efficiency from two complementary perspectives: online unlearning latency and offline CCI precomputation cost. Table V reports the end-to-end latency for a single unlearning request. CEU consistently achieves the lowest latency (0.34–6.37 s) across all datasets and backbones, outperforming the second-best baselines by $10.06\times$ to $55.64\times$. In contrast, full retraining requires 141.92–4209.05 s, and other baselines remain one to three orders of magnitude slower. This highlights CEU's real-time efficiency, achieved entirely through parameter-free score adjustments.

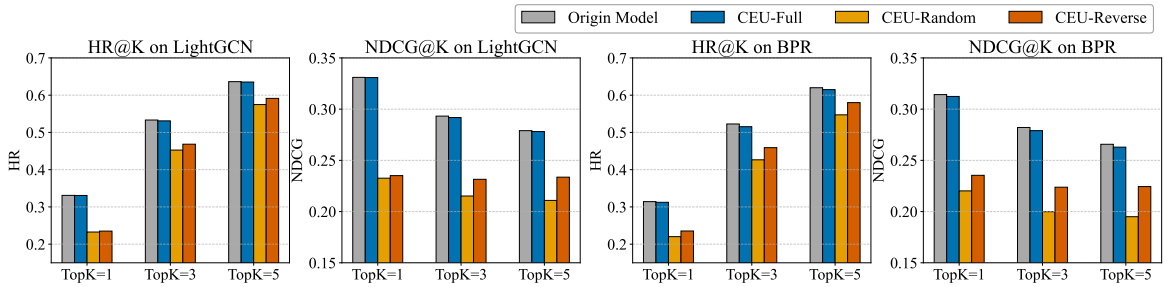


Fig. 4. Comparison of CEU variations on ML-1M with LightGCN and BPR backbones. CEU-Full preserves HR@20 and NDCG@20 nearly identical to the original model, whereas CEU-Random and CEU-Reverse cause substantial performance degradation.

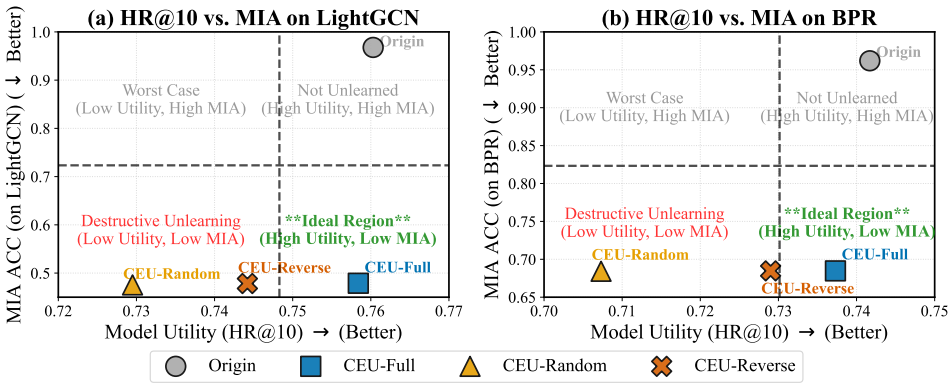


Fig. 5. Trade-off between recommendation utility and unlearning effectiveness on ML-1M under LightGCN and BPR. The bottom-right region is the ideal zone with high utility and low MIA accuracy. CEU-Full lies within this ideal zone, demonstrating precise, non-destructive forgetting, whereas CEU-Random and CEU-Reverse significantly degrade utility and lower MIA accuracy.

To ensure transparency, Table IV details the optional offline CCI precomputation cost. Using a single NVIDIA RTX 4090 GPU, we cache the Top-50 candidates (with up to five causal factors per item) for each user. The per-user computation time is modest (3.10–10.10 s), yielding a compact cache footprint of approximately 22 KiB that can be amortized across future requests.

For large-scale deployments, CEU can also operate on demand. Executing the CCI step only when a user actually submits an unlearning request reduces the need for full precomputation and storage while preserving the same online unlearning mechanism. Overall, these results confirm that CEU meets the efficiency goal (G2) and supports low-latency unlearning in practice.

E. Effectiveness of CCI (RQ4)

To validate the Causal Contribution Identifier (CCI) module, we conducted an ablation study on the ML-1M dataset comparing our full method (**CEU-Full**) against two non-causal variants: **CEU-Random** (applying random penalties) and **CEU-Reverse** (penalizing causally unrelated items). As shown in Fig. 4 and Fig. 5, **CEU-Full** almost perfectly preserves the original model's utility (e.g., LightGCN's NDCG@10 drops by a mere 0.4%, from 0.2682 to 0.2671), whereas the non-causal variants trigger a catastrophic performance collapse (e.g., **CEU-Random**'s NDCG@10 plummets by 19.4% on LightGCN and 20.8% on BPR). Furthermore, while these non-causal variants also reduce MIA accuracy to ≈0.48, this low MIA score is a false positive due to the complete collapse of the model's ranking ability. Ultimately, these results demonstrate that the CCI module is indispensable; it provides the precise causal guidance needed to achieve precise unlearning without sacrificing overall model utility.

VI. Conclusion

In this paper, we addressed recommendation unlearning under the strict asymmetric access constraints of MLaaS architectures, where parameter updates are prohibitive. To tackle the lack of interpretability and side-effect propagation in existing methods, we proposed CEU, a novel parameter-free "Explain, then Forget" framework. CEU first identifies minimal causal support sets via CCI and then performs localized score adjustment via URA to remove the influence of forgotten interactions without modifying model parameters. Extensive experiments across three benchmarks verify that CEU preserves recommendation utility close to that of the original model while significantly outperforming existing unlearning baselines in both forgetting quality and efficiency. The ablation study further confirms that causal guidance is essential for avoiding destructive side effects and maintaining model fidelity. Overall, CEU offers a practical solution for efficient and reliable recommendation unlearning in API-only MLaaS environments.

References

[1] M. Ribeiro, K. Grolinger, M. A. Capretz, Mlaas: Machine learning as a service, in: 2015 IEEE 14th international conference on machine learning and applications (ICMLA), IEEE, 2015, pp. 896–902.

[2] I. Grigoriadis, E. Vrochidou, I. Tsiatsiou, G. A. Papakostas, Machine learning as a service (mlaas)—an enterprise perspective, in: Proceedings of International Conference on Data Science and Applications: ICDSA 2022, Volume 2, Springer, 2023, pp. 261–273.

[3] R. Philipp, A. Mladenow, C. Strauss, A. Völz, Machine learning as a service: Challenges in research and applications, in: Proceedings of the 22nd International Conference on Information Integration and Web-based Applications & Services, 2020, pp. 396–406.

[4] Z. Luo, J. Yin, H. Yan, R. Liu, M. Cheng, S. Wang, One-turn knock-out: Traceable and editable proxy unlearning under asymmetric access constraints (2026).

[5] N. Sritrakool, S. Maneeroj, Personalized preference drift aware sequential recommender system, IEEE Access 9 (2021) 155491–155506.

[6] J. Chen, B. Wang, Z. Ouyang, et al., Personal recommender system based on user interest community in social network model, Physica A: Statistical Mechanics and its Applications 526 (2019) 120961.

[7] N. Hariri, B. Mobasher, R. Burke, Adapting to user preference changes in interactive recommendation., in: IJCAI, Vol. 15, 2015, pp. 4268–4274.

[8] Y. Li, X. Feng, C. Chen, Q. Yang, A survey on recommendation unlearning: Fundamentals, taxonomy, evaluation, and open questions, IEEE Transactions on Knowledge and Data Engineering 38 (2) (2025) 781–799.

[9] P. Lubitzsch, O. Ovcharenko, H. Chen, M. de Rijke, S. Schelter, Towards a real-world aligned benchmark for unlearning in recommender systems, arXiv preprint arXiv:2508.17076 (2025).

[10] S. Shen, C. Zhang, Y. Zhao, A. Bialkowski, W. Chen, M. Xu, Machine unlearning for streaming forgetting, arXiv preprint arXiv:2507.15280 (2025).

[11] L. Jiaqi, Z. Jiahao, L. Linjing, D. D. Zeng, Harmonizing supervised fine-tuning and reinforcement learning with reward-based sampling for continual machine unlearning, Electronics 15 (4) (2026) 771.

[12] M. Chen, Z. Zhang, T. Wang, M. Backes, M. Humbert, Y. Zhang, Graph unlearning, in: 2022 ACM SIGSAC, 2022, pp. 499–513.

[13] C. Chen, F. Sun, M. Zhang, B. Ding, Recommendation unlearning, in: ACM Web Conference, 2022, pp. 2768–2777.

[14] J. Cheng, G. Dasoulas, H. He, C. Agarwal, M. Zitnik, Gnndelete: A general strategy for unlearning in graph neural networks, in: Proceedings of the International Conference on Learning Representations (ICLR), 2023, pp. 1–12.

[15] G. Chen, L. Xia, C. Huang, Pre-training for recommendation unlearning, in: SIGIR, 2025, pp. 2112–2121.

[16] Y. Li, C. Chen, X. Zheng, Y. Zhang, B. Gong, J. Wang, L. Chen, Selective and collaborative influence function for efficient recommendation unlearning, EsWA 234 (2023) 121025.

[17] Y. Zhang, Z. Hu, Y. Bai, J. Wu, Q. Wang, F. Feng, Recommendation unlearning via influence function, ACM TORS 3 (2) (2024) 1–23.

[18] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, N. Papernot, Machine unlearning, in: 2021 IEEE SP, IEEE, 2021, pp. 141–159.

[19] H. Zhang, C. Zhang, P. Hu, S. Qiu, J. Wang, Customized retrieval-augmented generation with llm for debiasing recommendation unlearning, in: 2025 IEEE International Conference on Data Mining (ICDM), 2025, pp. 1702–1711. doi:10.1109/ICDM65498.2025.00183.

[20] Y. Xian, Z. Fu, S. Muthukrishnan, G. De Melo, Y. Zhang, Reinforcement knowledge graph reasoning for explainable recommendation, in: Proceedings of the 42nd international ACM SIGIR conference on research and development in information retrieval, 2019, pp. 285–294.

[21] Y. Xian, Z. Fu, H. Zhao, Y. Ge, X. Chen, Q. Huang, S. Geng, Z. Qin, G. De Melo, S. Muthukrishnan, et al., Cafe: Coarse-to-fine neural symbolic reasoning for explainable recommendation, in: Proceedings of the 29th ACM International Conference on Information & Knowledge Management, 2020, pp. 1645–1654.

[22] J. Tan, S. Xu, Y. Ge, Y. Li, X. Chen, Y. Zhang, Counterfactual explainable recommendation, in: CIKM '21, 2021, pp. 1784–1793.

[23] J. Tan, S. Geng, Z. Fu, Y. Ge, S. Xu, Y. Li, Y. Zhang, Learning and evaluating graph neural network explanations based on counterfactual and factual reasoning, in: WWW, 2022, pp. 1018–1027.

[24] J. Pearl, Causal inference, Causality: objectives and assessment (2010) 39–58.

[25] C. Rasmussen, Z. Ghahramani, Occam's razor, NeurIPS 13 (2000).

[26] S. Rendle, C. Freudenthaler, Z. Gantner, L. Schmidt-Thieme, Bpr: Bayesian personalized ranking from implicit feedback, arXiv:1205.2618 (2012).

[27] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, M. Wang, Lightgcn: Simplifying and powering graph convolution network for recommendation, in: SIGIR, 2020, pp. 639–648.

[28] Y.-M. Tamm, R. Damdinov, A. Vasilev, Quality metrics in recommender systems: Do we calculate metrics consistently?, in: RecSys, 2021, pp. 708–713.

[29] Y. Wang, L. Wang, Y. Li, D. He, T.-Y. Liu, A theoretical analysis of ndcg type ranking measures, in: Conference on learning theory, PMLR, 2013, pp. 25–54.

[30] R. Shokri, M. Stronati, C. Song, V. Shmatikov, Membership inference attacks against machine learning models, in: IEEE SP, IEEE, 2017, pp. 3–18.

[31] Y. Li, C. Chen, X. Zheng, J. Liu, J. Wang, Making recommender systems forget: Learning and unlearning for erasable recommendation, Knowledge-Based Systems 283 (2024) 111124.

Appendix

Stability under Larger Forget Ratios

To further examine the stability of different unlearning methods, we evaluate retained-set performance under increasing forget ratios. Specifically, we vary the forget ratio from 2% to 10% and normalize the retained-set performance of each method by its value at the 2% forget ratio. Thus, the vertical axis reports the relative retention ratio. A value closer to 1.0 indicates better preservation of retained-set utility as more interactions are forgotten. We note that this experiment is not a strict simulation of sequential request streams. Instead, it serves as a stress test of retained-set stability under stronger forgetting. As shown in Figure 6, most baseline methods show a clear downward trend as the forget ratio increases, although the degree of degradation varies across methods. In contrast, CEU remains much more stable across all settings and stays close to the no-update reference.

This result is consistent with the design of CEU. Since CEU performs unlearning via score adjustment rather than repeated parameter updates, it is less sensitive to utility degradation during repeated or large-scale forgetting. Therefore, this experiment provides supplementary evidence that the parameter-free design of CEU helps preserve retained-set utility as the forgetting intensity increases.

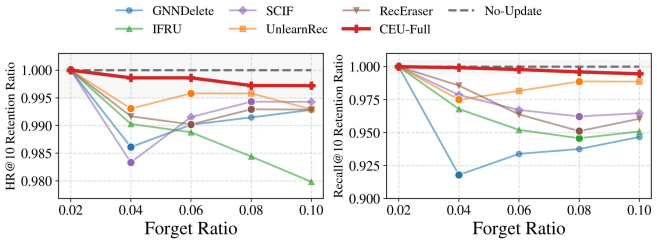


Fig. 6. Retained-set stability under increasing forget ratios. The left panel shows the HR@10 retention ratio, and the right panel shows the Recall@10 retention ratio, both normalized by the value at the 2% forget ratio. Higher values indicate better retained-set stability under stronger forgetting.

Algorithm 1: Causal Contribution Identifier (CCI)

Input: pretrained recommender $f(\cdot \mid \theta)$; user u; target item i; user history $\mathcal{H}_u$; top-K cutoff K; margin m; sparsity weight λ; threshold δ
Output: causal explanation $E_{ui} = (\mathcal{S}^*_{ui}, w_{ui})$
Initialize mask vector $\mathbf{P}_u \leftarrow \mathbf{1}$;
Obtain the original top-K list for user u and let i_{K+1} be the item ranked at position $K+1$;
repeat
  Construct the masked history $\mathcal{H}_u \odot \mathbf{P}_u$;
  Compute the flip loss

$$\mathcal{L}_{\text{flip}} = \max\left(0,\ m + f(u^{(\mathbf{P}_u)}, i \mid \theta) - f(u, i_{K+1} \mid \theta)\right)$$

  Update $\mathbf{P}_u$ by minimizing

$$\mathcal{L}_{\text{flip}} + \lambda \|\mathbf{1} - \mathbf{P}_u\|_1$$

until *convergence*;
Compute the causal support set

$$\mathcal{S}^*_{ui} = \{h \in \mathcal{H}_u \mid P_{u,h} < 1 - \delta\}$$

For each $h \in \mathcal{S}^*_{ui}$, compute its contribution strength

$$w_{ui}(h) = 1 - P_{u,h}$$

return $E_{ui} = (\mathcal{S}^*_{ui}, w_{ui})$;

Algorithm 2: Unlearning-driven Rank Adjuster (URA)

Input: pretrained recommender $f(\cdot \mid \theta)$; user u; forgotten interaction $h_f = (u, i_f)$; current recommendation list $\mathcal{L}_u$; cached explanations $\{E_{ui}\}_{i \in \mathcal{L}_u}$; penalty factor α
Output: updated recommendation list $\mathcal{L}'_u$
foreach $i \in \mathcal{L}_u$ **do**
  Compute the original score $\hat{y}_{ui} = f(u, i \mid \theta)$;
  Retrieve $E_{ui} = (\mathcal{S}^*_{ui}, w_{ui})$;
  if $h_f \in \mathcal{S}^*_{ui}$ **then**
    Adjust the score by

$$\hat{y}'_{ui} = \hat{y}_{ui} - \alpha \cdot \hat{y}_{ui} \cdot w_{ui}(h_f)$$

  else
    Set $\hat{y}'_{ui} = \hat{y}_{ui}$;
Re-rank items in $\mathcal{L}_u$ according to $\{\hat{y}'_{ui}\}$;
Obtain the updated recommendation list $\mathcal{L}'_u$;
return $\mathcal{L}'_u$;

Pseudo Algorithms of CEU

This appendix summarizes the main CEU procedure in algorithmic form. CEU consists of two stages. First, the **Causal Contribution Identifier (CCI)** explains a recommendation by identifying the historical interactions that causally support it and by estimating their contribution strengths. Second, the **Unlearning-driven Rank Adjuster (URA)** uses these causal signals to remove the influence of a forgotten interaction by adjusting scores, without modifying model parameters.

In practice, CCI can be run offline to pre-compute explanations for candidate items and cache them for later use. When an unlearning request arrives, URA reads the cached explanations and updates the ranking scores online. This design makes CEU suitable for low-latency unlearning in API-only environments.

Experiment Setup Details

A. Evaluation Metrics

We evaluate recommendation utility using two standard ranking metrics: Hit Ratio (HR@K) and Normalized Discounted Cumulative Gain (NDCG@K) [28], [29]. Higher values indicate better recommendation quality on the retained data after unlearning. We evaluate unlearning completeness using Membership Inference Attack (MIA) accuracy [30]. This metric measures how well an attacker can distinguish between forgotten and retained interactions. An MIA accuracy closer to 0.5 means the forgotten interactions are harder to identify, indicating better forgetting. We evaluate unlearning efficiency by recording the end-to-end latency of unlearning requests. Lower latency indicates stronger efficiency.

B. Implementation Details

All experiments are conducted on a server with one NVIDIA RTX 4090 GPU. We fix the random seed to 2020 for all runs. Unless otherwise stated, all models are trained with Adam for 300 epochs, using a learning rate of 0.001 and a training batch size of 2048. We evaluate the model on the validation set after each epoch and apply early stopping with a patience of 10, where Recall@10 is used as the validation metric. For evaluation, we adopt the full-ranking setting and report Recall, NDCG, and Hit at $K \in 10, 20$, with an evaluation batch size of 4096.

For CEU, the CCI module pre-computes causal explanations for the top 50 candidate items of each user. For each candidate item, we retain at most 5 causal interactions as its explanatory support set. These cached explanations are then used by URA during the unlearning stage to perform score adjustment without updating model parameters.